Softwares de Sistemas
Aula – Software de Sistemas

SOFTWARE BÁSICO, TURMA A

PROF. MARCELO LADEIRA – CIC/UNB

Ligador
Introdução

Ligador (do inglês, *Linker*)

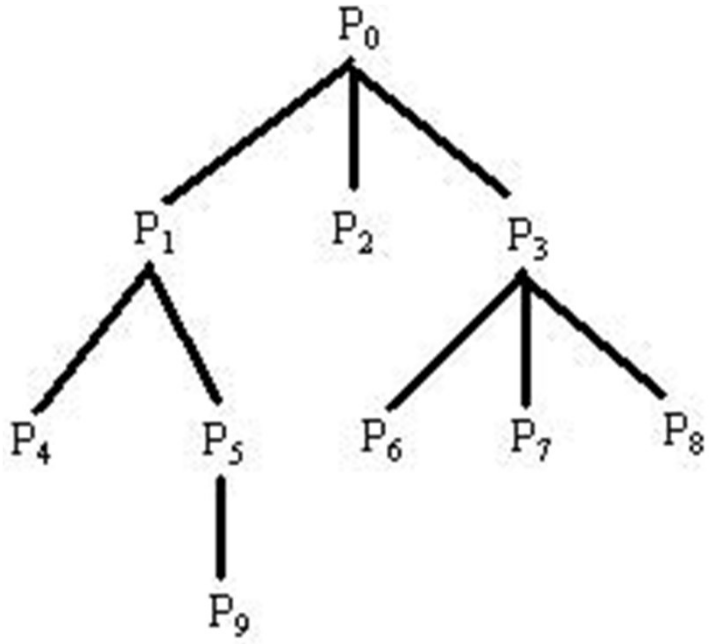
- Programa que resolve as referências externas em um programa.
- Entrada
  - Arquivo objeto com código relocável.
- Saída
  - Arquivo binário com código de máquina relocável executável
  - Endereços relocáveis
- Função básica
  - Ligação de módulos
  - Resolução de referências externas

Introdução
Exemplo de código Assembly

```
extrn   sub1:far
cseg segment para public 'code'
start  proc near
        .
        call sub1
        .
        .
        call sub1
start  endp
cseg   ends
        end
```

```
subseg segment para public 'code'
sub1   proc far
        public sub1
        .
        .
        retf
sub1   endp
subseg ends
end
```

Introdução
Árvore estática de um programa



Primeiros tradutores produziam programas objeto na forma de código de máquina absoluto

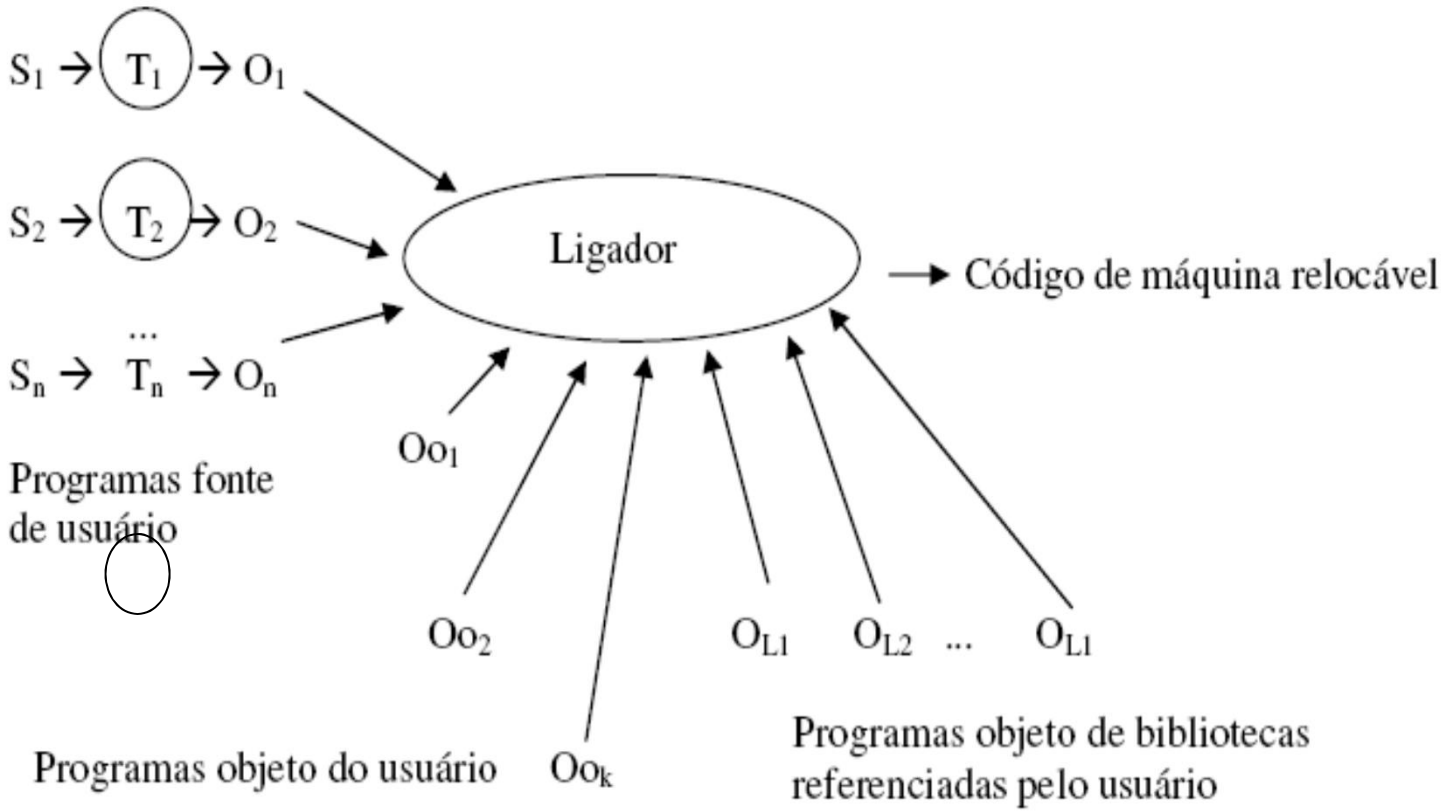
- tradução de todos P_i simultaneamente
  - bibliotecas com programa fonte

Atualmente tradutores produzem programas objeto relocáveis

- contêm informação de ligação simbólica.

REQUER O CONHECIMENTO PRÉVIO DA OCUPAÇÃO DA MEMÓRIA PARA SE PODER USAR ENDEREÇOS ABSOLUTOS.

Introdução: representação de um ligador



Introdução

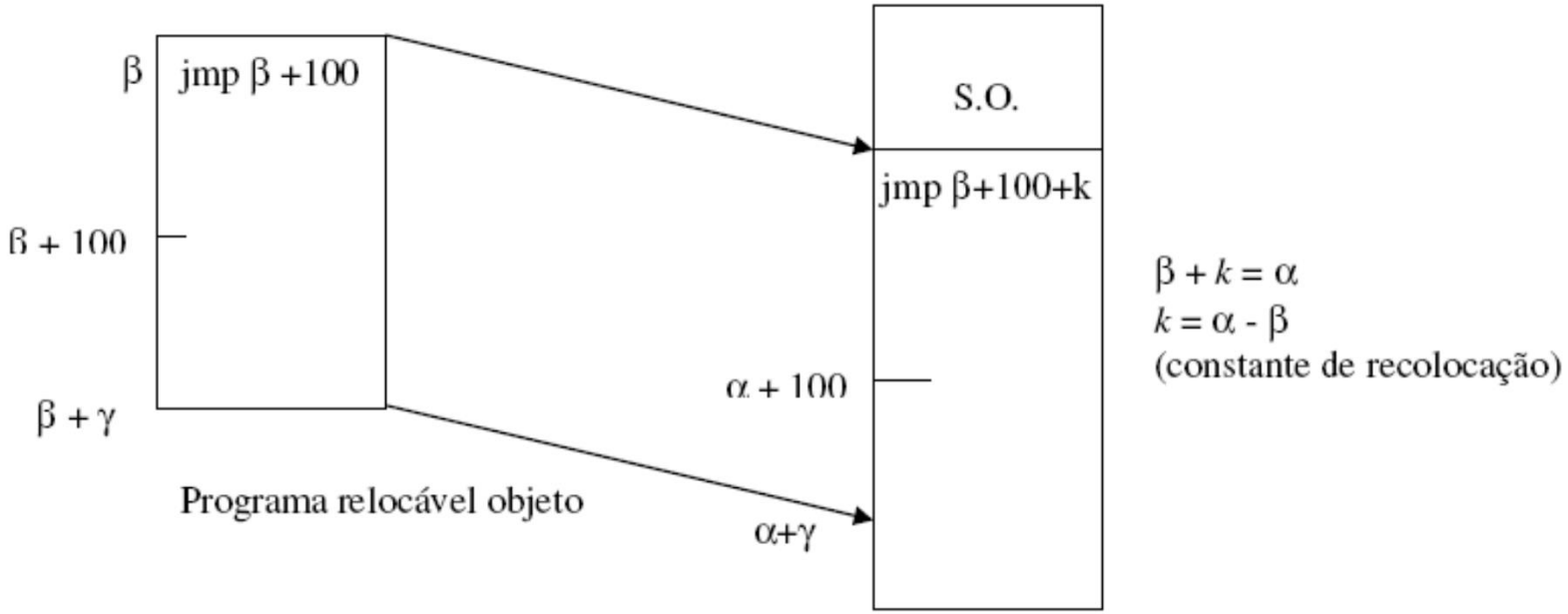
Relocação estática

Não se sabe onde o programa será colocado na memória real até o tempo de carga.

- Tradutores usam endereços relativos a uma base inicial
  - Sequencia de instruções e dados relativa a base arbitrário β.
- Todo código fonte é relocado em tempo de tradução (isto é, antes da sua execução)

O código gerado será carregado em posições contíguas de memória a partir do endereço (α) fornecido pelo sistema operacional.

Introdução
Relocação Estática



Estruturação

Processo de ligação

- Programa ou segmento de dados
  - Símbolos internos
    - Definidos para possível referência por outros programas.
  - Símbolos externos
    - Referenciados neste módulo mas definidos em outro módulo

Etapas

- Ler e relocar o módulo objeto P.
- Atualizar a tabela global de símbolos com os símbolos internos de P que podem ser referenciados em outros módulos.
- Concatenar P aos módulos anteriores.
- Resolver as referências externas em P.

Estruturação

Divisão Lógica

- Relocação de todo o programa objeto.
- Resolução das referências externas.

Número de passos

- Grosseiramente, os ligadores são classificados pelo número de passos:
  - leituras do arquivo objeto

Estruturação Lógica

Ligador de um passo

- Maior complexidade devido ao problema da resolução de referências externas sem antes ter-se montado a tabela global de símbolos

Ligador de dois passos

- Passo 1: relocação

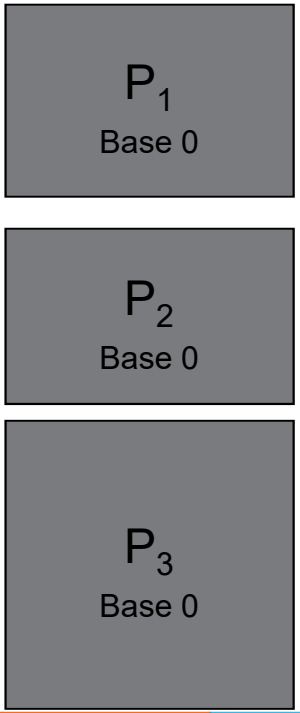
  obtém P, calcula k e reloca P considerando *k*.
  Inclui símbolos públicos de P na tabela de símbolos globais.
  Verifica validade do *entry-point*.

- Passo 2: resolução de referências externas

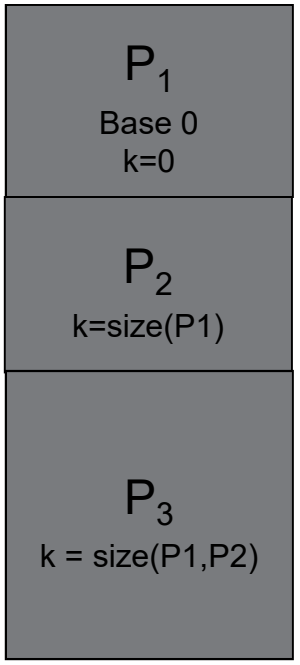
  obtém referências externas de P.
  Resolve as referência externa de P consultando a tabela global de símbolos.

Estruturação Lógica

Programas separados



Programa ligado



Principais Estruturas de Dados

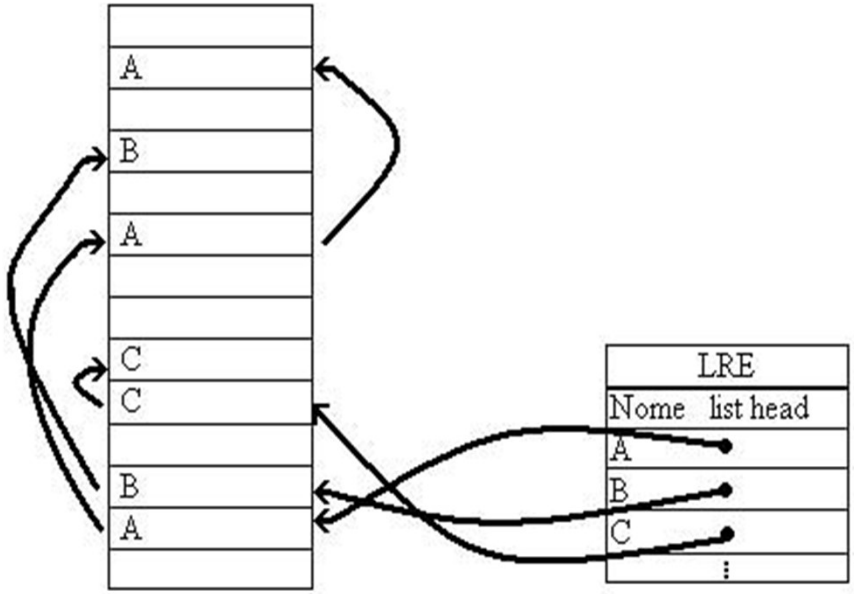
Código objeto

- Estruturado em seções com registros de tamanho variável.
  - Dependente da arquitetura de hardware e do desenvolvedor do sistema operacional.
    - O formato objeto da Microsoft possui registros (seções) especiais que o da Intel não possui.
    - No Dev-Cpp a opção `-c` gera um arquivo com código objeto, com extensão .o
      - o default é o ligador ser chamado após o compilador e o arquivo .o ser apagado.

Principais Estruturas de Dados

Lista de referências externas

- Contém os símbolos referenciados mas não declarados no programa



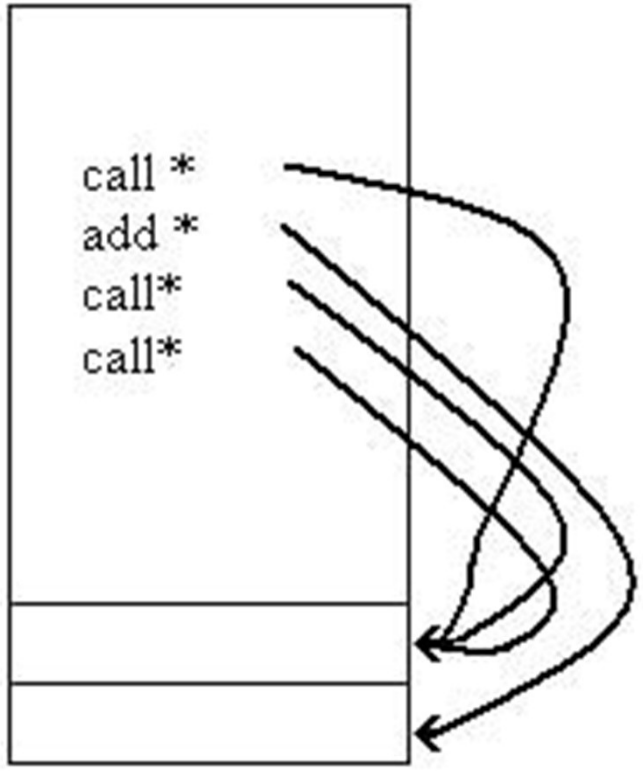
FIXUPP CONTÉM UM TABELA COM OS ENDEREÇOS RELOCÁVEIS, INCLUINDO OS EXTERNOS

Principais Estruturas de Dados

Abordagem alternativa

Vetor de transferência

- Associa a cada símbolo externo um endereço local onde armazenar o endereço real do símbolo.
  - Uso de endereçamento indireto.
  - Quando o símbolo for definido, seu valor é inserido na posição dada pelo endereçamento indireto
- Ligação é rápida.
- Execução requer dois acessos à memória ao invés de apenas um para referenciar um símbolo externo



Principais Estruturas de Dados

Tabela de Símbolos Global

- Armazena os símbolos públicos definidos em cada módulo
  - Estruturação típica
    - Identificador
    - Tipo
    - Valor (endereço)
    - Valor inicial
  - Sugestão: árvore binária de pesquisa

Tipos de Ligadores

Linkage Editor

- Executa a ligação antes da carga.
- Após a ligação o programa é gravado em um arquivo.
  - O programa resultante é chamado de módulo de carga.
- Na carga é necessário apenas relocar o código.
  - A única alteração é a dos endereços relativos.
- Mais eficiente se um programa precisa ser executado diversas vezes.
- Na manutenção não é necessário recompilar todo o programa.

Linking Loader ou binder

- Após a ligação o programa é carregado na memória.
- Um *linking loader* em geral possui 2 passos:
  - atribui endereços aos símbolos externos
  - executa a carga, realocação e ligação.
- Não é gerado um arquivo executável.
- Se o programa precisa ser alterado com frequência, o uso de um *linkage Editor* é mais eficiente.

Carregador de Relocação

Carrega na memória RAM um programa executável contido em disco

- reloca-o para adequar o espaço de endereçamento ao endereço de carga fornecido pelo sistema operacional.

Em Ambiente mono-usuário:

- Lê o tamanho do programa a carregar e o entrypoint.
- Solicita ao gerente de memória a alocação de um bloco contíguo de memória com tamanho suficiente para conter todo o programa .
- Carrega o programa a partir do endereço dado pelo sistema operacional, relocando os endereços relocáveis.
- Transfere o controle para o programa carregado, a partir do entrypoint.

Tipos de Ligadores

Dynamic linking

- A ligação ocorre em tempo de execução
- A rotina é carregada e ligada ao restante do programa quando for chamada pela primeira vez
  - Possibilita que diversos programas compartilhem uma mesma cópia de uma rotina em memória
  - Rotinas podem ser carregadas apenas se forem realmente necessárias
- A carga das rotinas necessárias é realizada em colaboração com o sistema operacional

NO AMBIENTE MICROSOFT EXISTEM AS BIBLIOTECAS DLL PARA LIGAÇÃO DINÂMICA.

Teoria de Biblioteca Dinâmica:

Bibliotecas dinâmicas, ao contrário de bibliotecas estáticas, não tem seu código incluído dentro do arquivo objeto. O código se mantém em arquivos separados no disco.

Quando se usa esse tipo de biblioteca o ligador apenas mantém um registro de que bibliotecas e quais funções se usam e somente depois, em tempo de carregamento ou de execução é que esse código é acessado pelo carregador (loader).



Teoria de Biblioteca Dinâmica:

O carregador é quem se encarrega de resolver a relocação das funções de bibliotecas usadas dentro do programa.

Isso acontece com o uso de uma tabela de relocação, chamada de fixup, que contém os endereços das bibliotecas e que é atualizado em tempo de execução pelo carregador.



Biblioteca de ligação dinâmica:

No Windows DLL é uma sigla para Dynamic Link Library.

- Normalmente possuem as extensões .dll, .ocx (bibliotecas contendo controles ActiveX), .drv (para drivers legados) ou .cpl (do painel de controle do Windows). Em casos mais raros uma dll pode ser .icl e .fon ou .fot, sendo bibliotecas de ícones e fontes respectivamente.

No GNU/Linux e Solaris:

- Os nomes dos arquivos sempre começam com lib e terminam com .so (shared object), opcionalmente com o número da versão da interface.



Teoria sobre DLL:

DLLs tem esse nome porque são carregadas pelo ligador (linker) dinamicamente, ou seja, em tempo de execução.

Pode-se fazer isso de duas formas:

a normal que carrega todas as DLLs associadas a um programa quando esse for carregado,

ou usando de*layed loading*, na qual a DLL somente é carregada quando uma chamada para alguma função é feita.

A segunda forma possui a vantagem de poder tratar exceções, dado que na primeira forma se o SO não encontrar uma DLL necessária o programa nem é carregado e na segunda o SO retorna exceção durante a execução.



Teoria sobre DLL:

As vantagens de se usar DLLs são as seguintes:

- Permite o compartilhamento por vários programas de uma única cópia de funções
- Promove arquitetura modular.
- Dá suporte a programas multilinguagem. Programas escritos em linguagens diferentes podem chamar a mesma DLL contanto que chamem a função da mesma forma.
- Facilita a criação de versões internacionais. Basta guardar as strings de cada língua em DLLs separadas e carregá-las quando precisar.



Mas e então, o que é na prática?

Imagine que você vá fazer um programa em C e esse programa usa uma certa função, digamos, “int wasGood (char *name)”

A função wasGood pode ser declarada em um arquivo de ligação dinâmica e ser importada na sua maneira específica.

Assim, essa função só se ligará ao programa em runtime e poderá ser usada por outros programas



Aplicação: DLL no Windows

No sistema operacional Windows, essas ligações se fazem por meio das DLLs (Dynamic-link library)

Usa-se um arquivo com extenção .dll onde foram declaradas as funções e se referencia no programa principal.



Exemplo: arquivo DLL

```
#include  <windows.h>
#include  <string.h>
extern “C” __declspec(dllexport) int wasGood (char *str);
BOOL APIENTRY
 DllMain(HANDLE hModule, DWORD dwReason, LPVOID lpReserved)
 {
     return TRUE;   }
int wasGood (char *str) {
     return (!strcmp(str, “Cruzeiro”));
}
```



Exemplo: programa principal

```
#include <windows.h>
#include <stdio.h>
extern “C” __declspec(dllimport) int wasGood (char *str);
int main (int argc, char **argv){
    printf(“Esse time %s eh bom? %s”, argv[1], wasGood(argv[1])
    ? “ sim” : “ nao”);
    return 1;
}
```

